SECURE DEV LABS
TASK 02

INTERNSHIP PROGRAM 2026

NETWORK SECURITY MONITORING

SURICATA IDS/IPS & ZEEK NSM

INSTRUCTOR: ARSHMAN ABBAS

REPORT BY: AHSAN ARFEEN SHAIKH

Suricata IDS Deployment and Rule Based Traffic Detection on Ubuntu

1. Introduction

Intrusion Detection Systems (IDS) are essential components of modern cybersecurity. They monitor network traffic to identify suspicious activities, malicious attacks, and policy violations. Suricata is an open source, high-performance IDS/IPS developed by the Open Information Security Foundation (OISF). It performs deep packet inspection, protocol identification, and signature-based detection in real time.

The objective of this lab was to install Suricata on Ubuntu, configure it to monitor the network interface, create custom detection rules, and verify that alerts were generated successfully whenever specific websites such as YouTube and Facebook were accessed.

2. Objectives

The objectives of this practical were:

- **Install Suricata IDS on Ubuntu.**
- **Configure Suricata using the default configuration file.**
- **Update and download the latest detection rules.**
- **Test the configuration.**
- **Run Suricata in live packet capture mode.**
- **Create custom detection rules.**
- **Detect YouTube traffic.**
- **Detect Facebook traffic.**
- **Monitor Suricata logs.**
- **Verify generated alerts.**

3. System Requirements

Operating System

Ubuntu 22.04 LTS

Virtualization Software

Oracle VirtualBox

RAM

2 GB

Processor

Dual Core

Network Adapter

NAT

Enp0s3

4. Installing Suricata

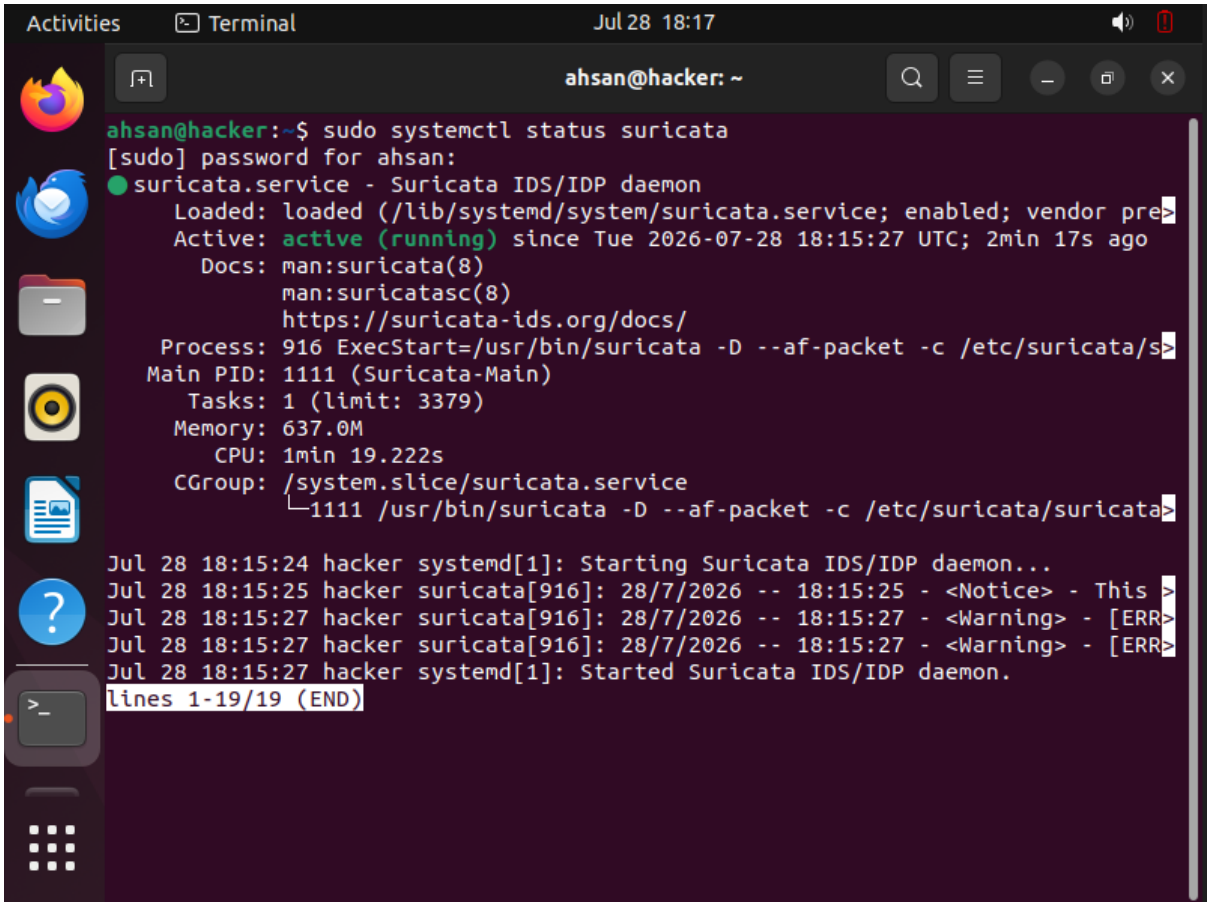
The first step was installing Suricata IDS on Ubuntu.

Command:

(Sudo apt update)

(sudo apt install suricata -y)

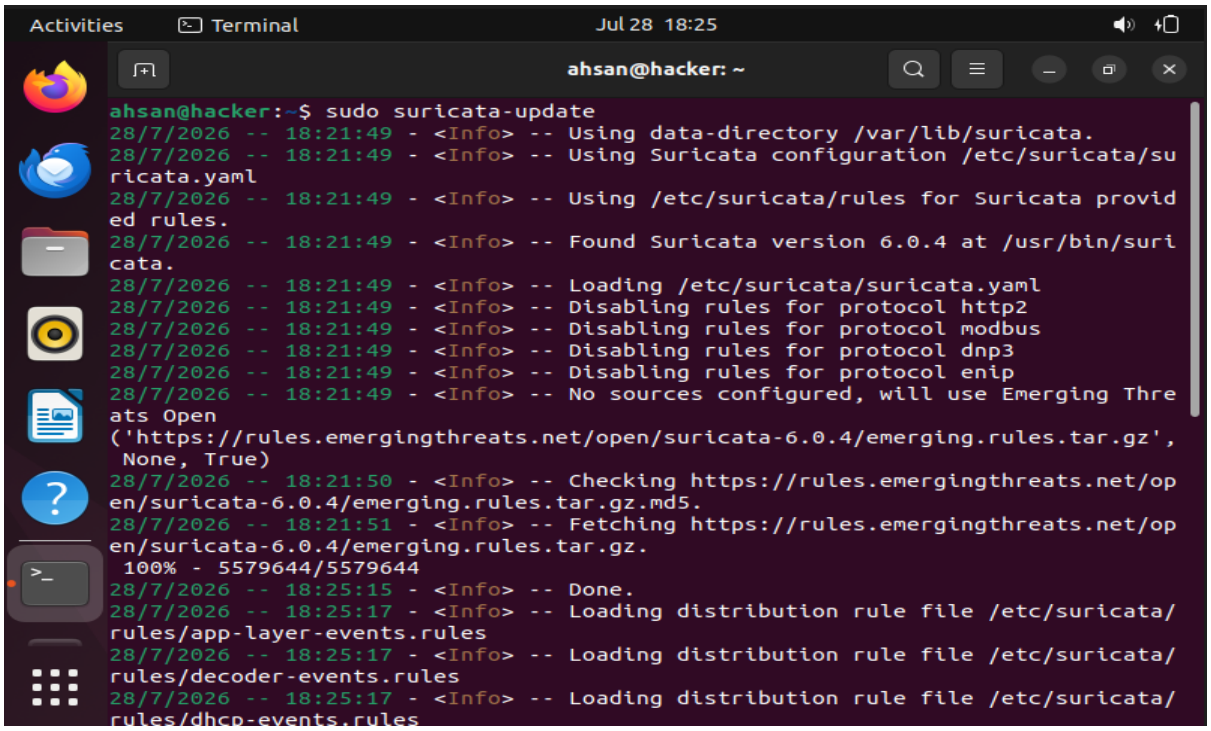
Below screenshots shows suricata is sucessfully installed and runing.

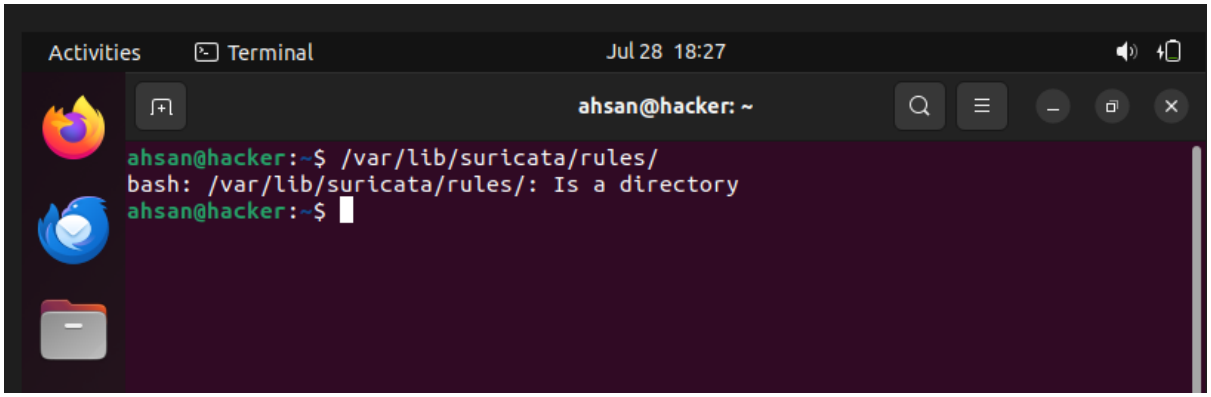


6. Updating Detection Rules

Command:

(Sudo suricata-update)





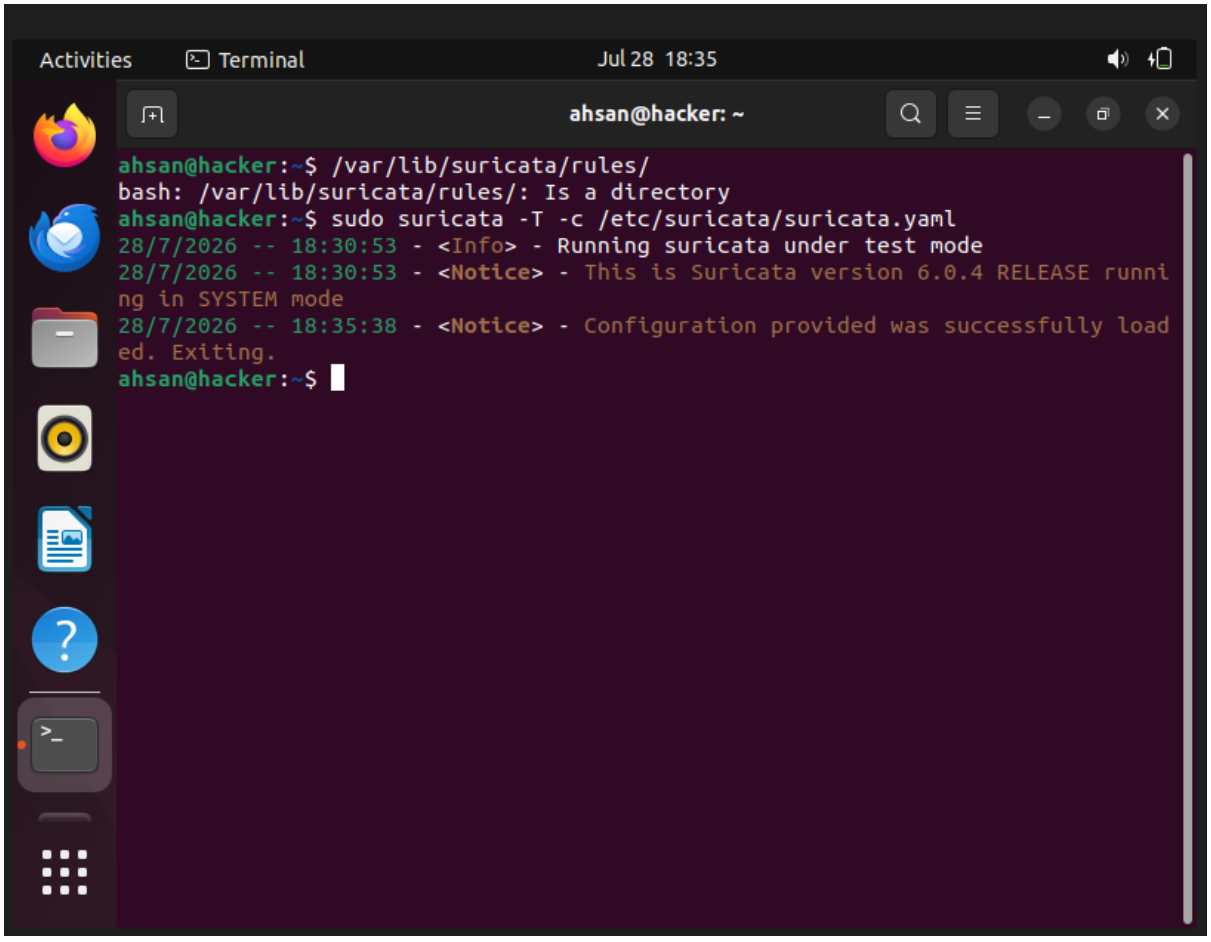
***SUCESSFULLY ADDED RULES**

7. Testing the Configuration

Command:

(sudo suricata -T -c /etc/suricata/suricata.yaml)

- **-T runs suricata in test mode**
- **It checks**
- **Syntax error**
- **Missing rules**
- **Configuration**



RESULTS: Configuration was provided sucessfully

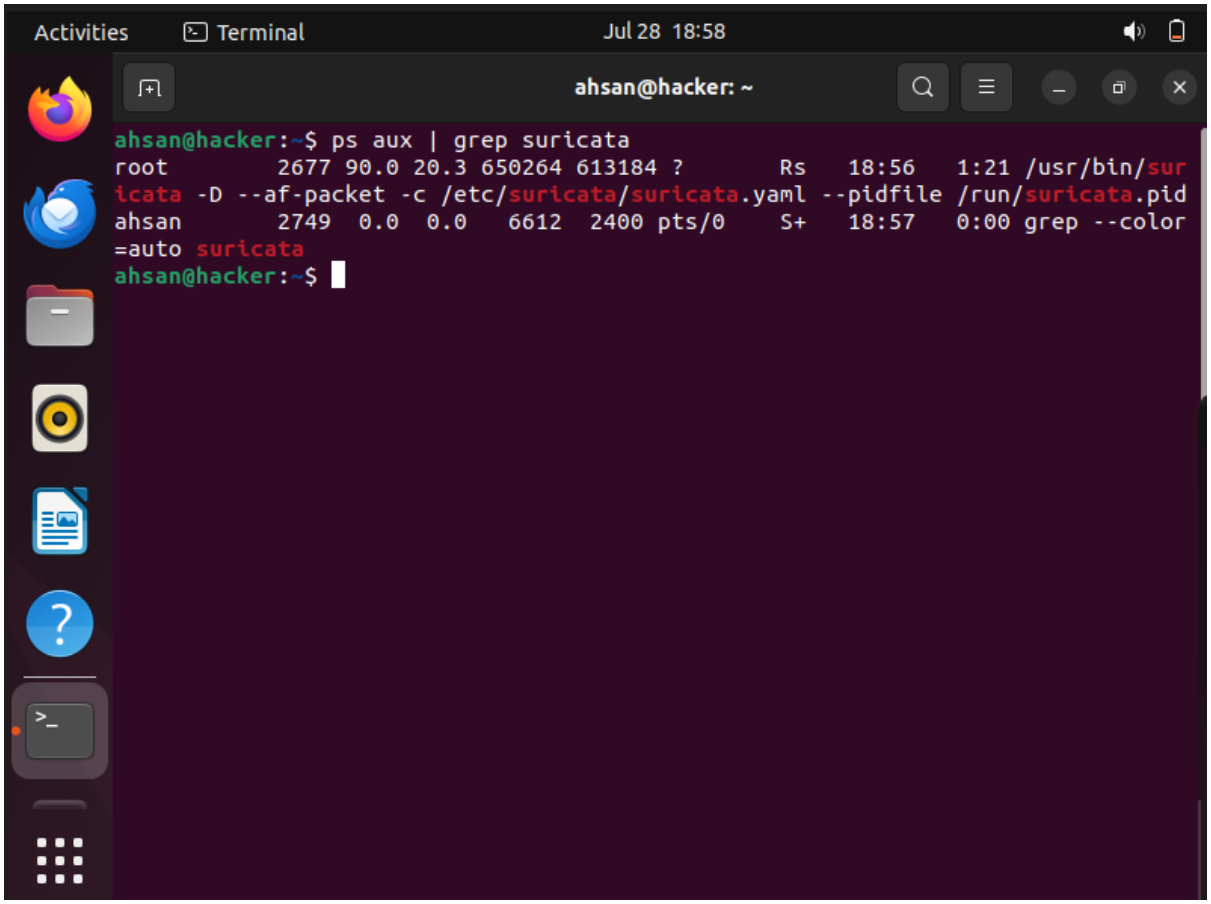
8. Starting Suricata

Command:

`(sudo suricata -c /etc/suricata/suricata.yaml -i enp0s3)`

Purpose

This command starts Suricata in IDS mode and captures packets from the network interface



- **Ps aux | grep suricata : shows the status that suricata is running**

09. Creating Custom Rules

The default rules detect many attacks. However, custom rules can also be written to detect organization-specific events.

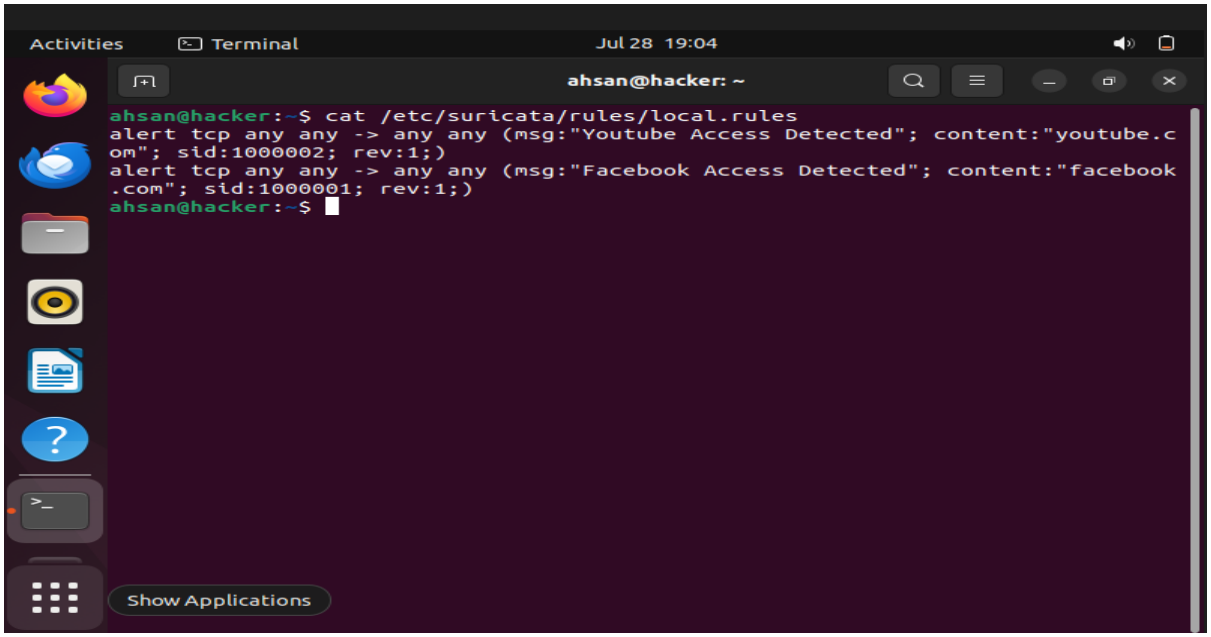
The local rule file is located at:

```
/etc/suricata/rules/local.rules
```

```
Rule for Youtube
```

```
alert http any any -> any any (msg:"YouTube Access Detected"; content:"youtube.com"; sid:1000002; rev:1
```

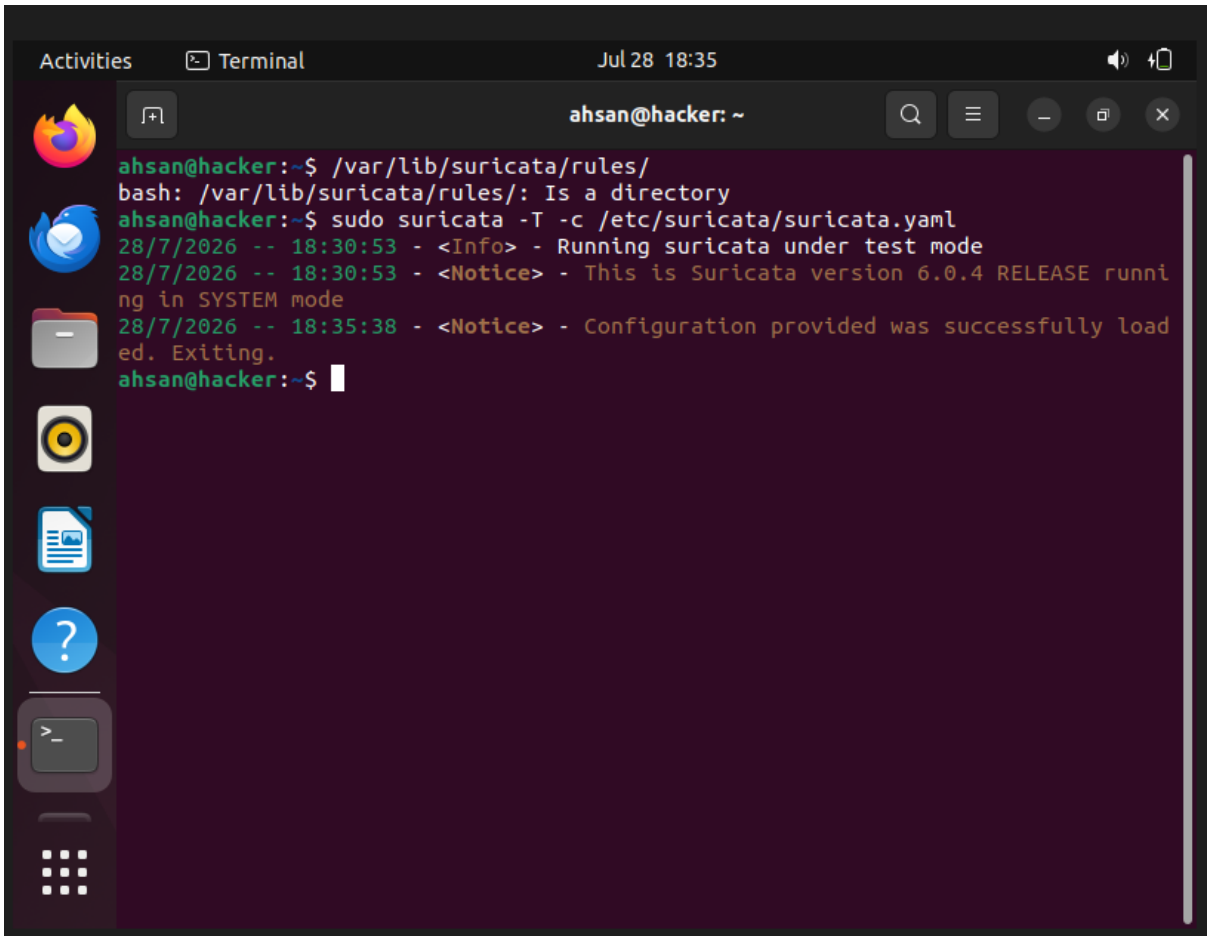
```
Rule for Facebook
alert http any any -> any any (msg:"Facebook Access Detected"; content:"facebook.com"; sid:1000002; rev:1
```



Sucessfullu added rules in local.rules

10.Reloading Configuration

After adding rules, Suricata configuration was tested again.



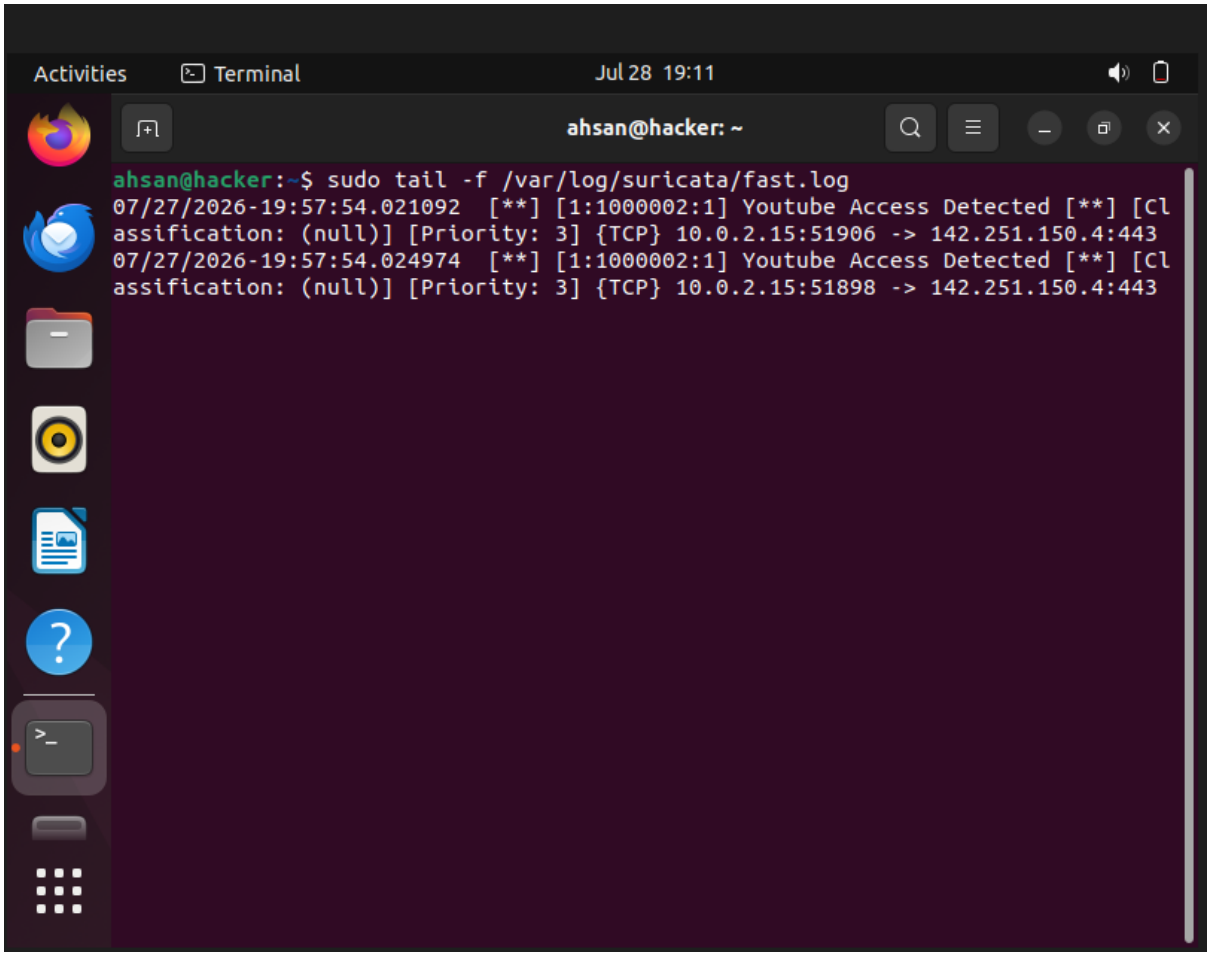
11. Monitoring Alerts

Command:

sudo tail -f /var/log/suricata/fast.log

Purpose:

Displays alerts in real time.



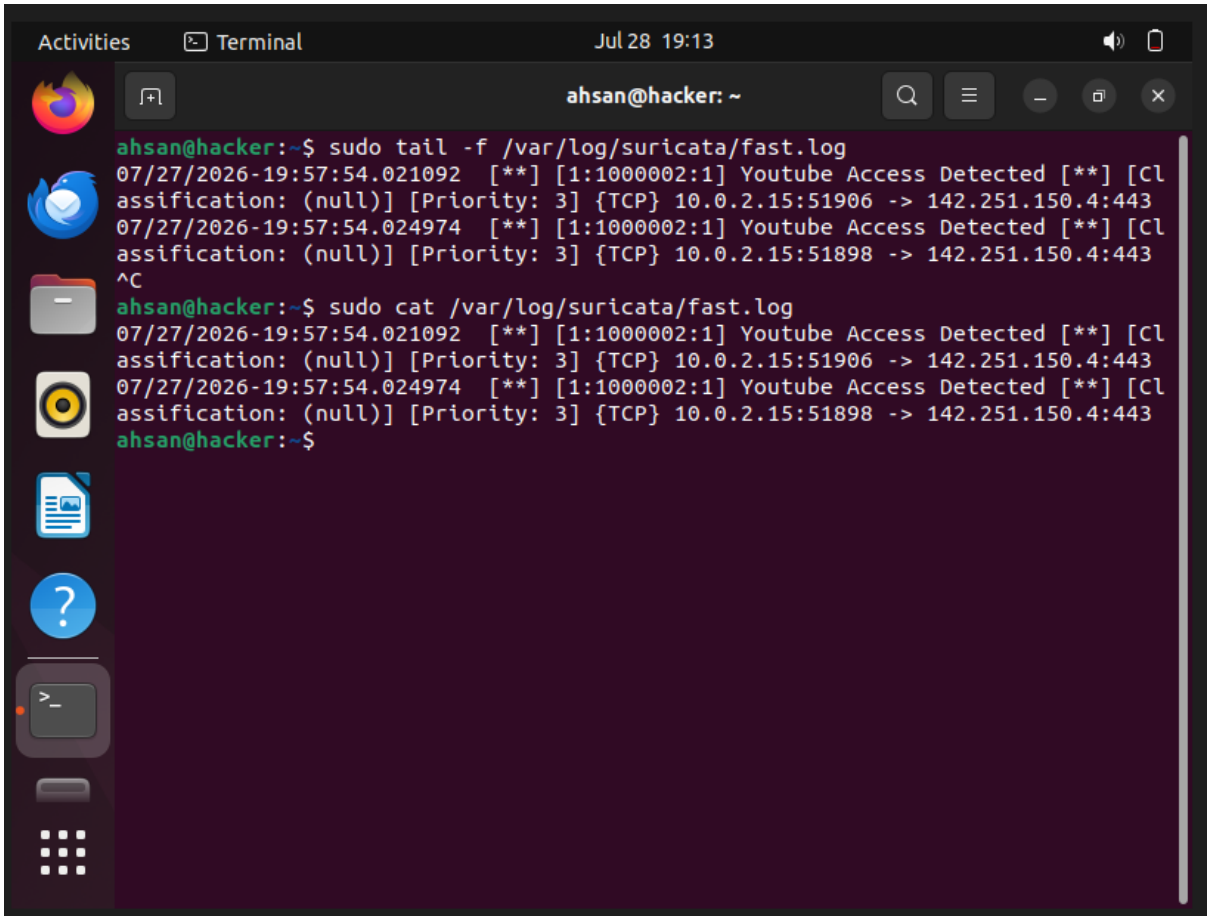
12. Viewing Complete Alert Logs

Command:

sudo cat /var/log/suricata/fast.log

Purpose:

Displays all generated alerts.



13. Parsing the EVE JSON Log

After generating alerts, Suricata stores detailed information in the eve.json log file. Unlike fast.log, which only displays brief alerts, eve.json contains complete event information in JSON format, including timestamps, source and destination IP addresses, protocols, alerts, and metadata. The jq utility is used to read this file in a human-readable format.

Steps: Installing jq

Command:

sudo apt install jq -y

Purpose:

This command installs the jq utility, which is used to read and process JSON files from the Linux terminal.

Steps: Displaying the Complete EVE JSON Log

Command:

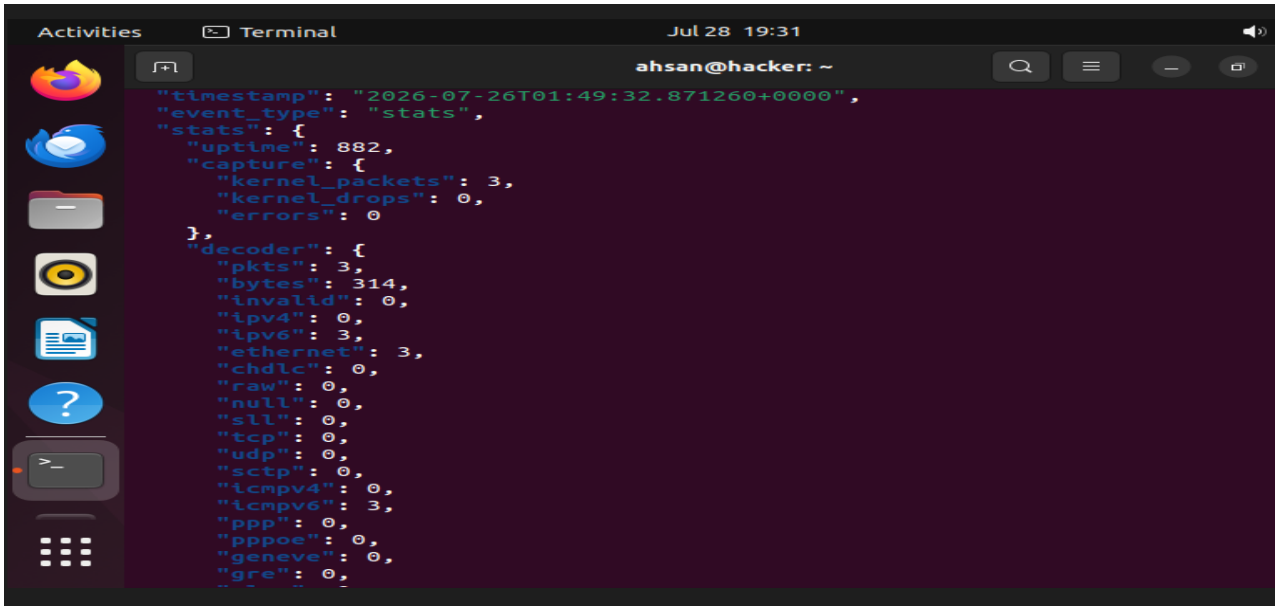
```
jq '.' /var/log/suricata/eve.json
```

Purpose:

This command displays all events stored in the eve.json file in a properly formatted and readable JSON structure.

Result:

The complete Suricata event log was displayed, including network traffic details, timestamps, protocols, and alert information.



Steps: Displaying Only Alert Events

Command:

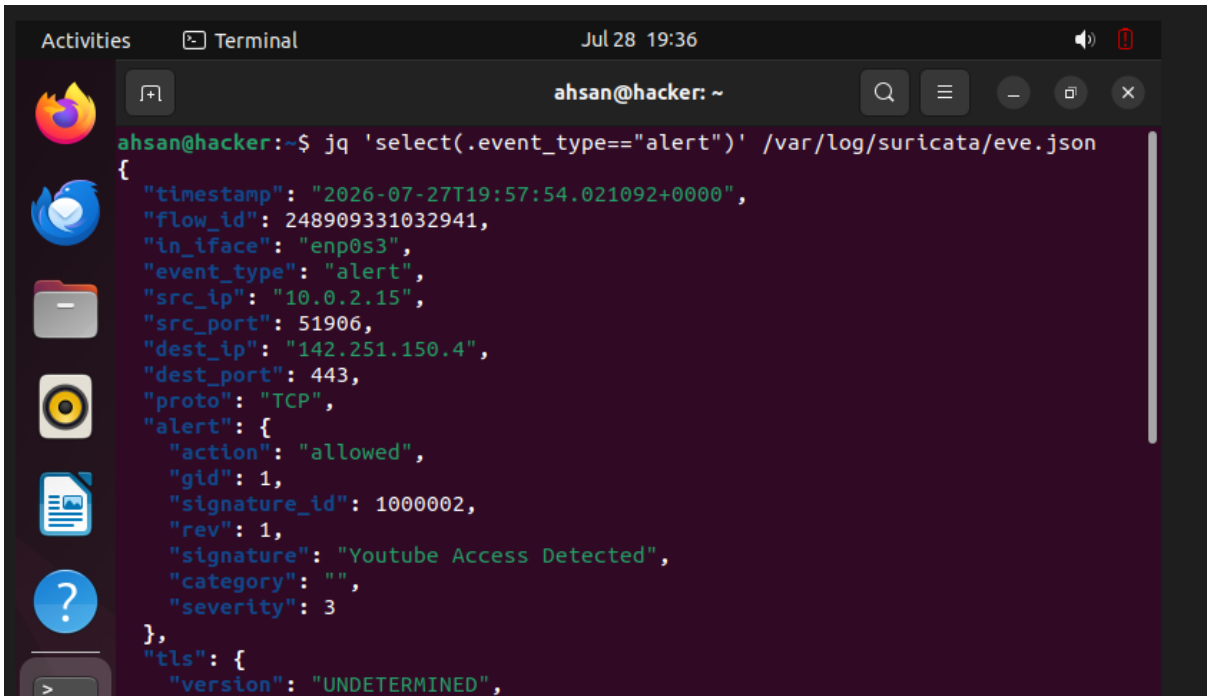
```
jq 'select(.event_type=="alert")'
/var/log/suricata/eve.json
```

Purpose:

This command filters the log file and displays only events whose type is alert, hiding all other event types such as DNS, HTTP, and flow records.

Result:

Only IDS alerts generated by Suricata were displayed



Steps: Displaying Only Alert Signatures

Command:

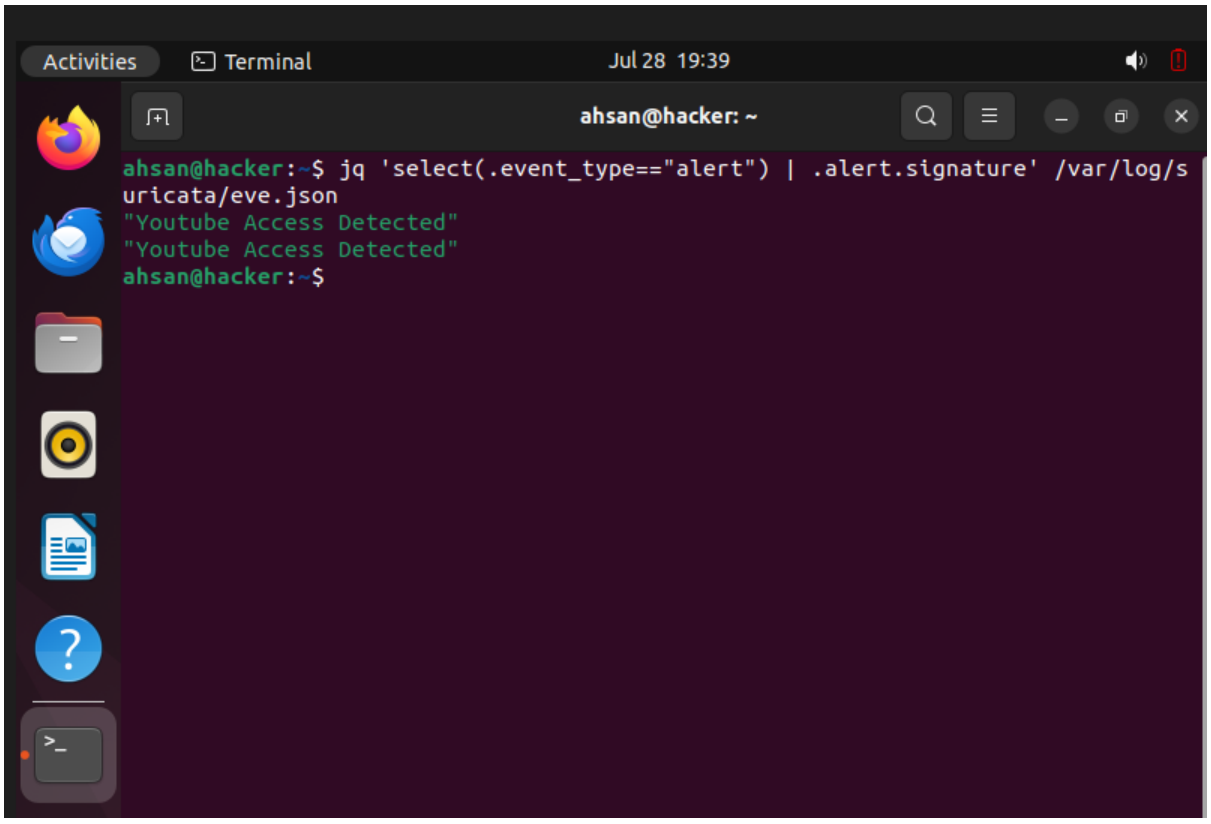
```
jq 'select(.event_type=="alert") | .alert.signature'
/var/log/suricata/eve.json
```

Purpose:

This command extracts only the alert messages (signatures) from the JSON log.

Example Output:

```
"YouTube Access Detected"
```



14. Automating Rule Updates

To ensure that Suricata always uses the latest threat detection rules, rule updates can be automated using the Linux cron scheduler.

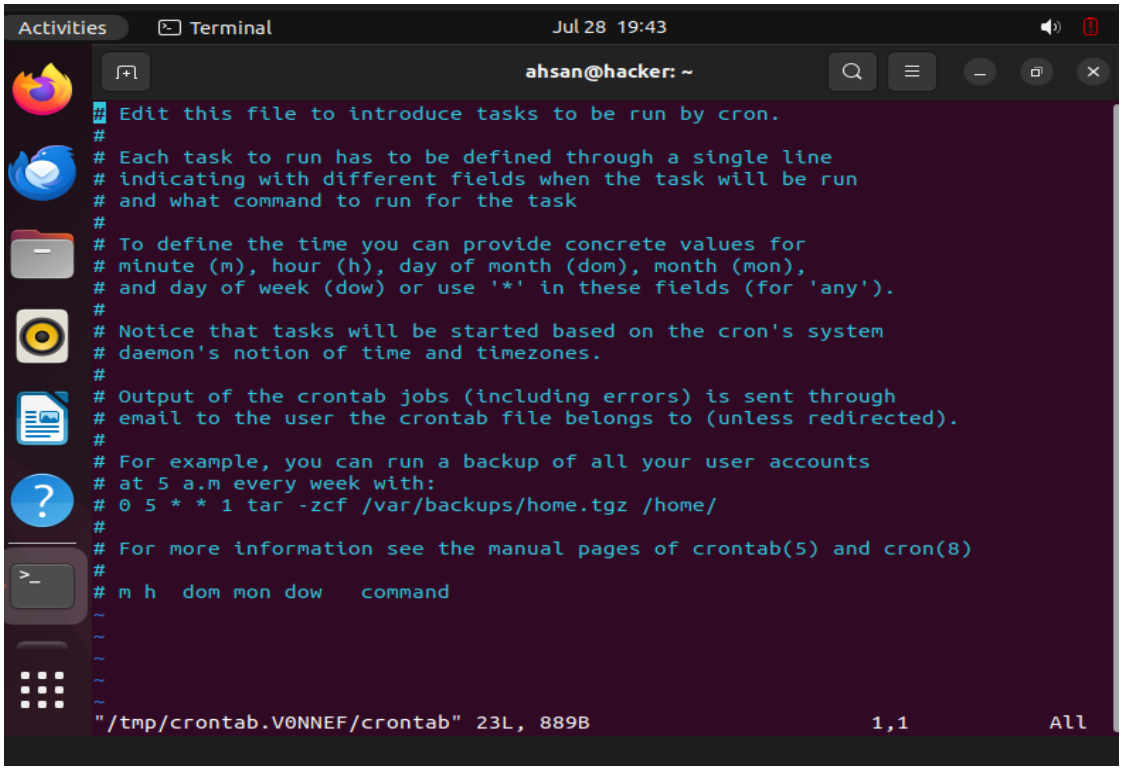
Steps: Opening the Cron Table

Command:

sudo crontab -e

Purpose:

This command opens the root user's cron table, allowing scheduled tasks to be created or modified.



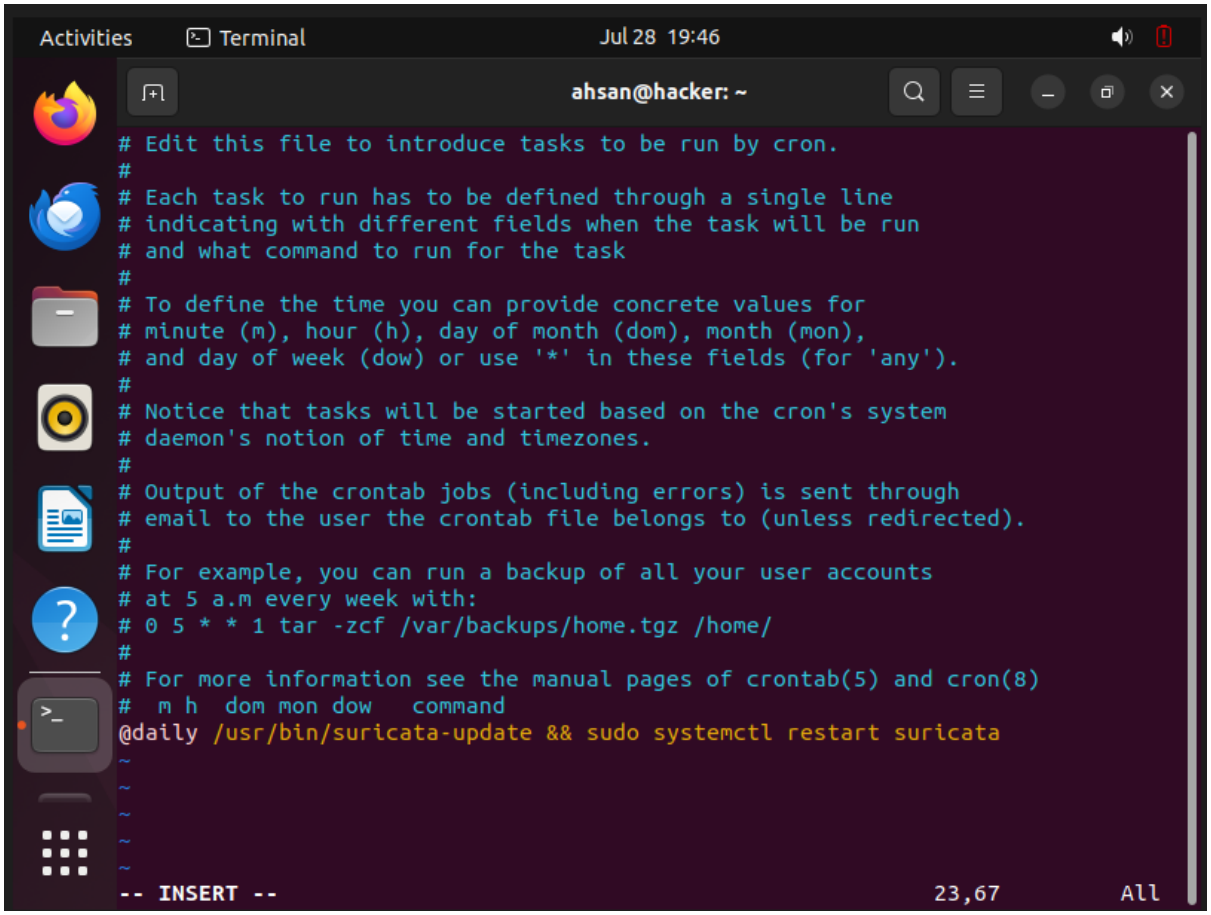
Steps: Scheduling Automatic Rule Updates

Add the Following Line:

```
@daily /usr/bin/suricata-update && systemctl restart suricata
```

Purpose:

This scheduled task performs the following actions every day:



15. Block Facebook Traffic Using IPS Mode

Objective:

Configure Suricata in Intrusion Prevention System (IPS) mode to block access to Facebook using a custom DROP rule.

Steps: Create a Facebook Blocking Rule

Command:

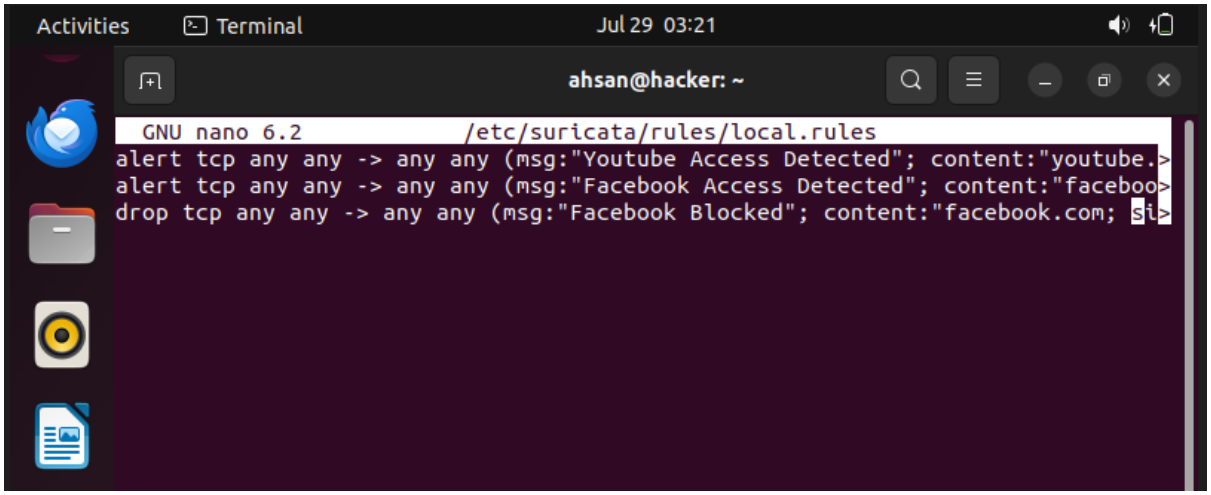
```
sudo nano /etc/suricata/rules/local.rules
```

Add the following rule:

```
drop tcp any any -> any any (msg:"Facebook Blocked";
content:"facebook.com"; sid:1000010; rev:1;)
```

Purpose:

Created a custom IPS rule to detect and block traffic containing the string facebook.com.



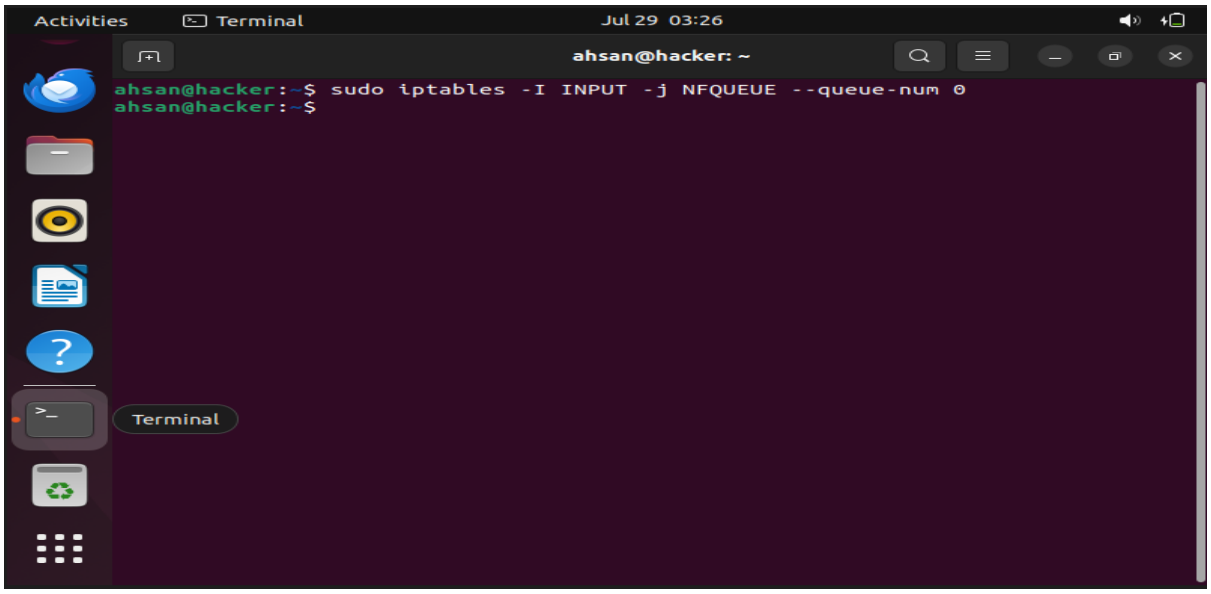
Steps: Configure NFQUEUE

Command:

```
sudo iptables -I INPUT -j NFQUEUE --queue-num 0
```

Purpose:

Configured Linux `iptables` to send incoming packets to NFQUEUE so that Suricata can inspect and block malicious traffic.



Result

Traffic was redirected to NFQUEUE successfully.

PART 2: Zeek — Network Security Monitor

. Introduction to Zeek

Zeek is a network security monitoring (NSM) framework. Unlike Suricata, which focuses on alerting, Zeek does not fire alerts by default instead, it silently records detailed structured logs of everything happening on the network. Suricata acts as an alarm system, while Zeek functions as a security camera system, capturing rich metadata for later forensic analysis.

Zeek processes packets from either a live interface or a saved PCAP file, identifies the application-layer protocol in use, extracts relevant metadata (IPs, ports, timestamps, byte counts, protocol-specific fields), and writes this information into structured, tab-separated log files such as `conn.log`, `http.log`, `dns.log`, and others.

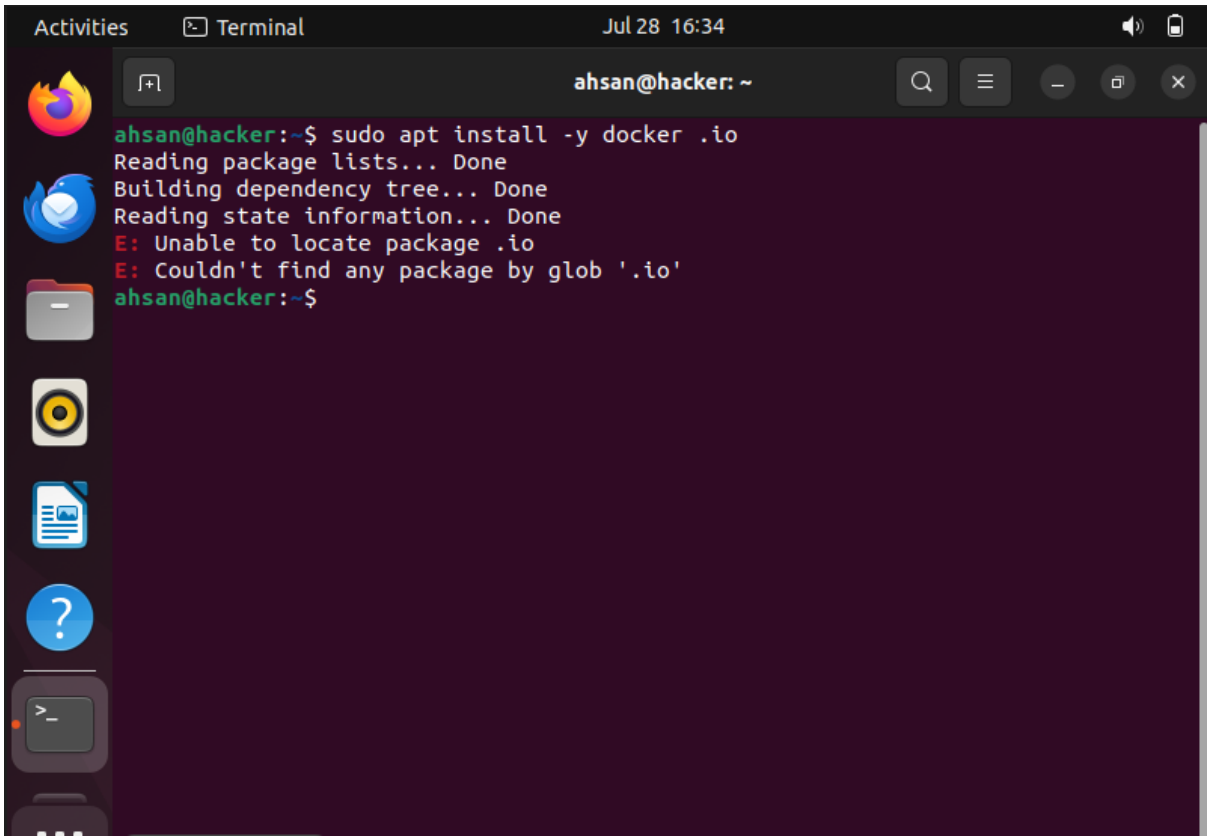
. Installing Zeek

The manual's standard installation method — adding the OpenSUSE Zeek repository via `apt` — failed repeatedly in this environment due to package resolution errors. As an approved alternative (Section 14 of the manual, "Zeek via Docker"), Zeek was installed and run using the official Zeek Docker image instead.

Step 1: Install Docker

Commands:

sudo apt update
sudo apt install -y docker.io



Step 3: Pull and run the Zeek container

Commands:

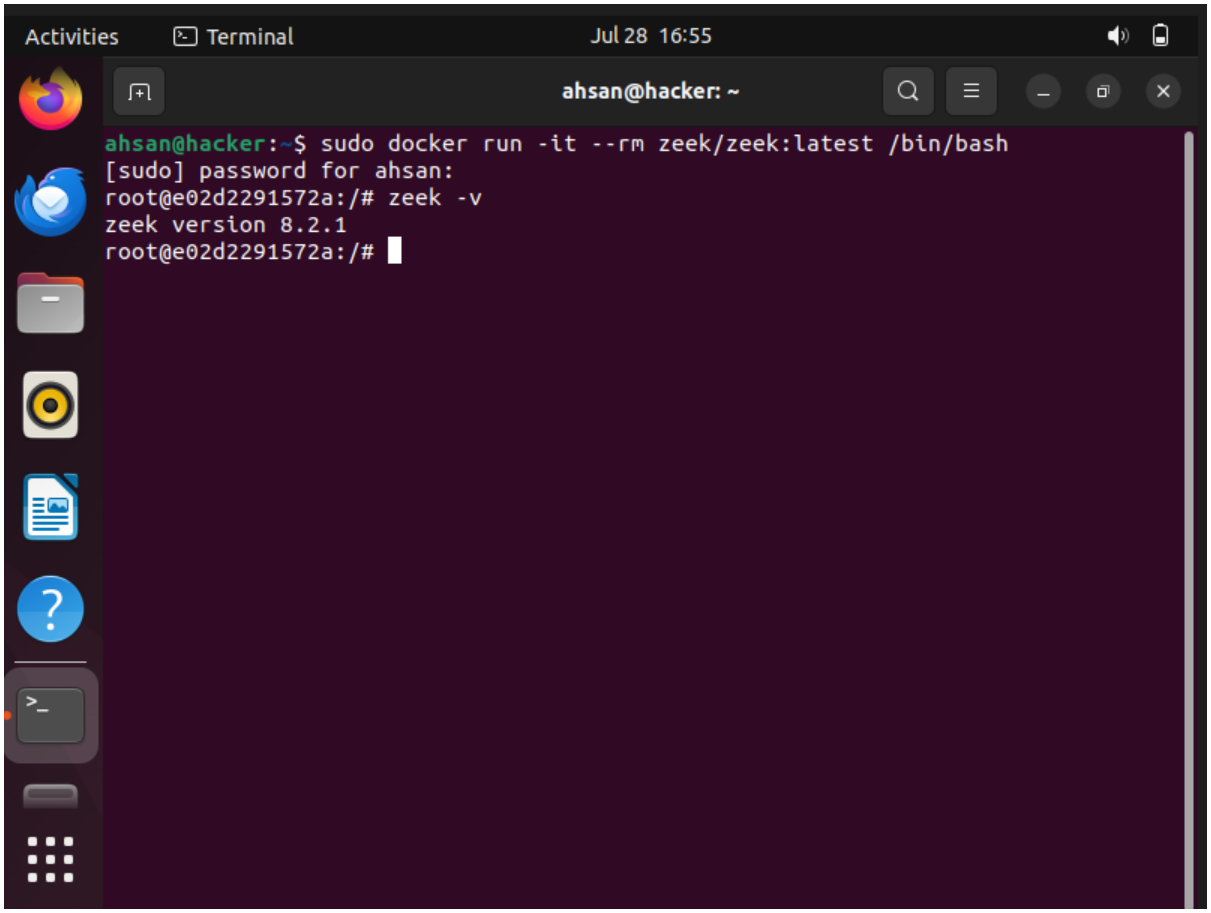
sudo docker pull zeek/zeek:latest
sudo docker run -it --rm --net=host zeek/zeek:latest /bin/bash

The `--net=host` flag was used so the container could see the VM's real network interface (`enp0s3`) directly, rather than being isolated on Docker's internal virtual network.

Step 4: Verify installation

Command:

zeek –v



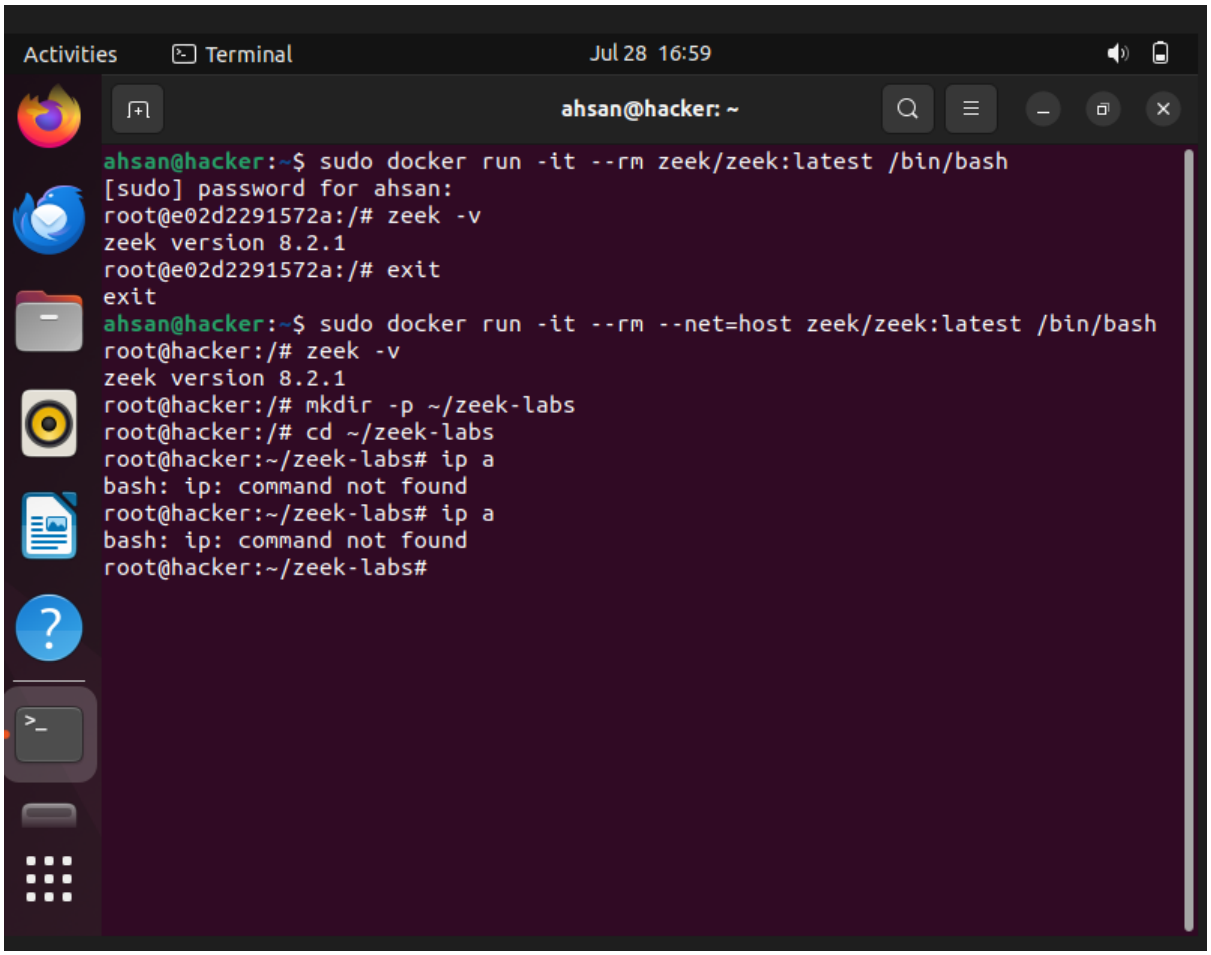
Step 5: Confirm host networking and create working directory

Since the Zeek Docker image is minimal, it does not include the `ip` command. Interface visibility was instead confirmed via:

cat /proc/net/dev

This confirmed `enp0s3` was visible inside the container.

mkdir -p ~/zeek-labs
cd ~/zeek-labs



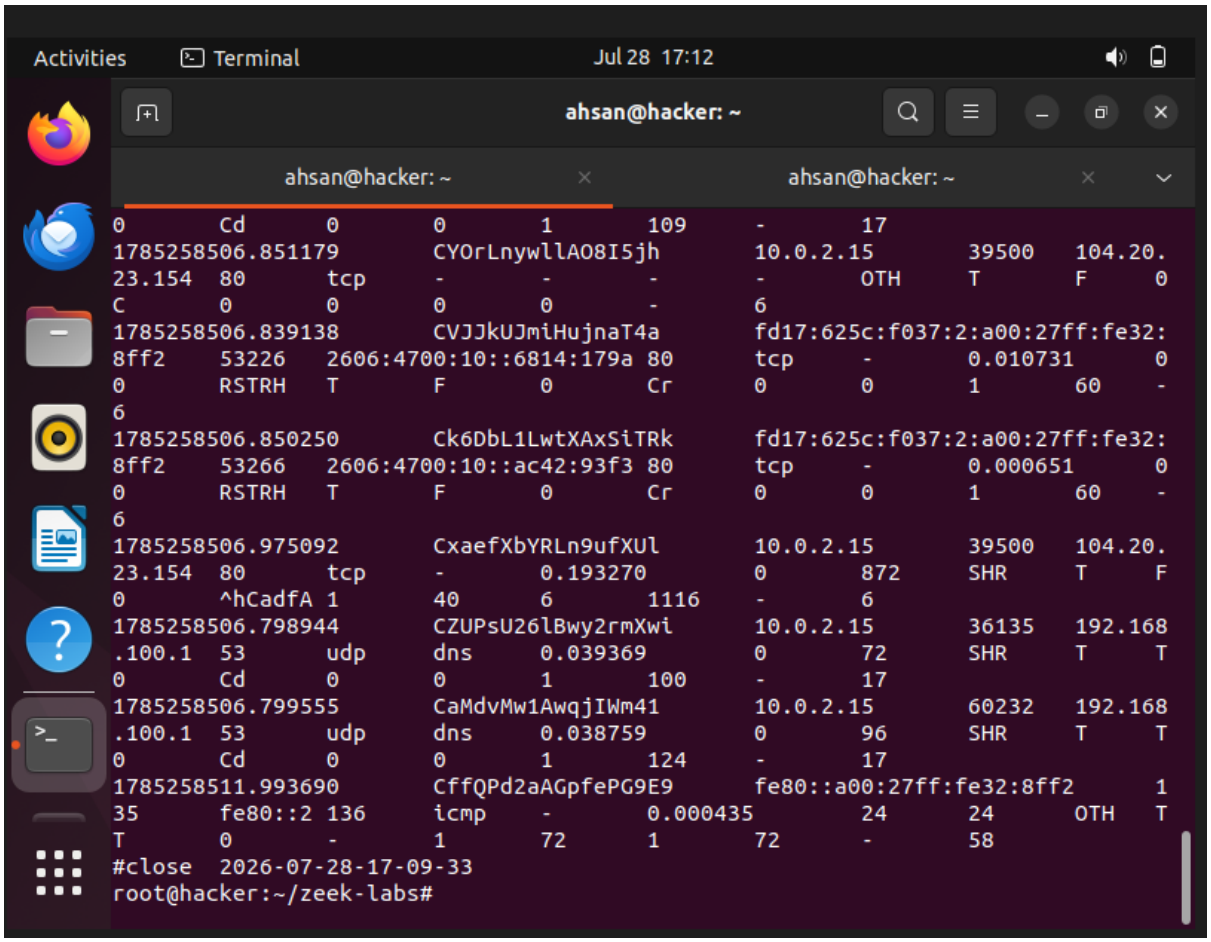
Task Z1: Generate and Analyze Web (HTTP) Traffic

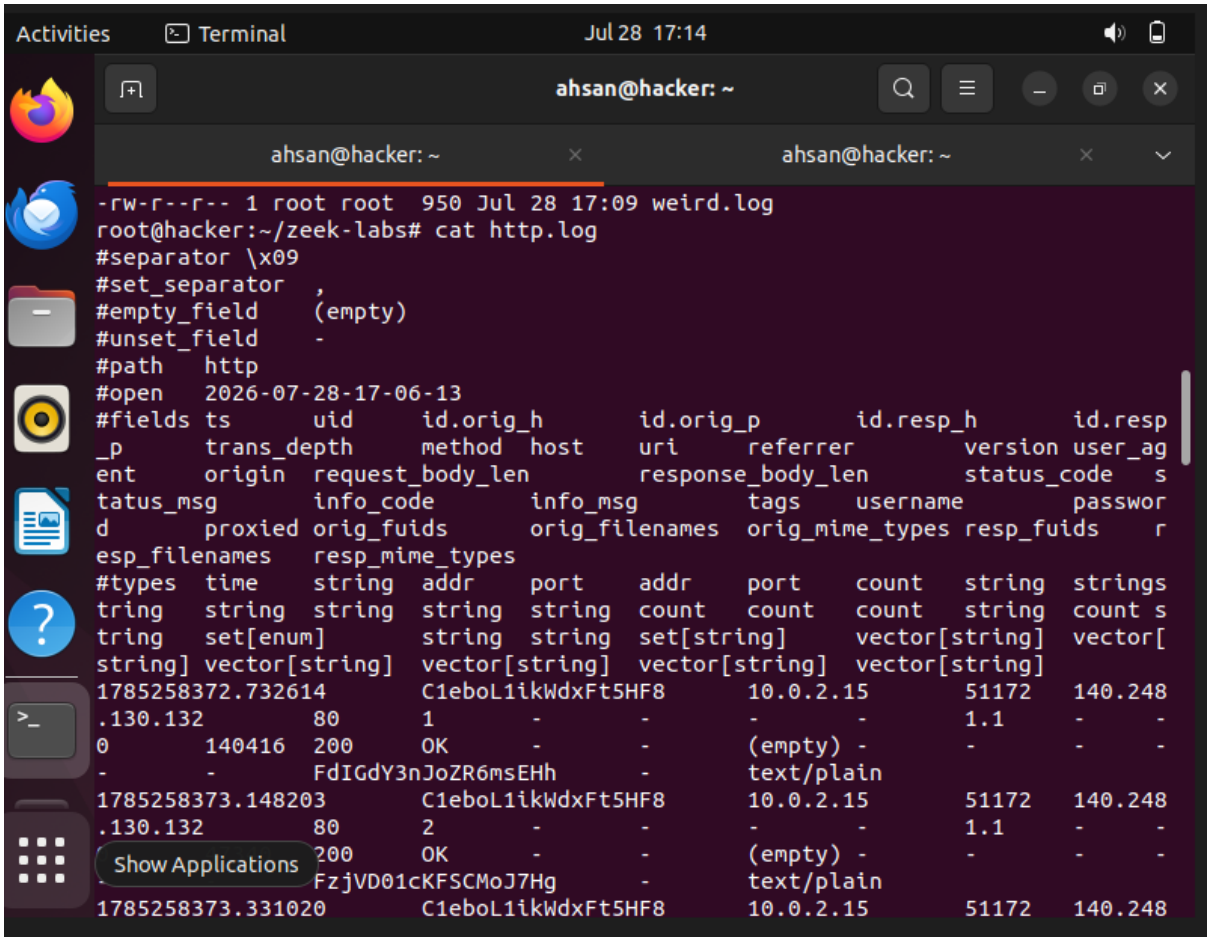
Zeek was started in live capture mode on `enp0s3`. In a second terminal attached to the same container via `docker exec`, HTTP traffic was generated:

curl http://example.com

Zeek was then stopped, and the logs inspected.

`conn.log` records every TCP/UDP/ICMP connection source/destination IPs and ports, protocol, duration, byte counts, and connection state. `http.log` records HTTP-specific details: method, host, URI, status code, and content type, cross-referenced to the same connection via a unique ID (UID).





Findings:

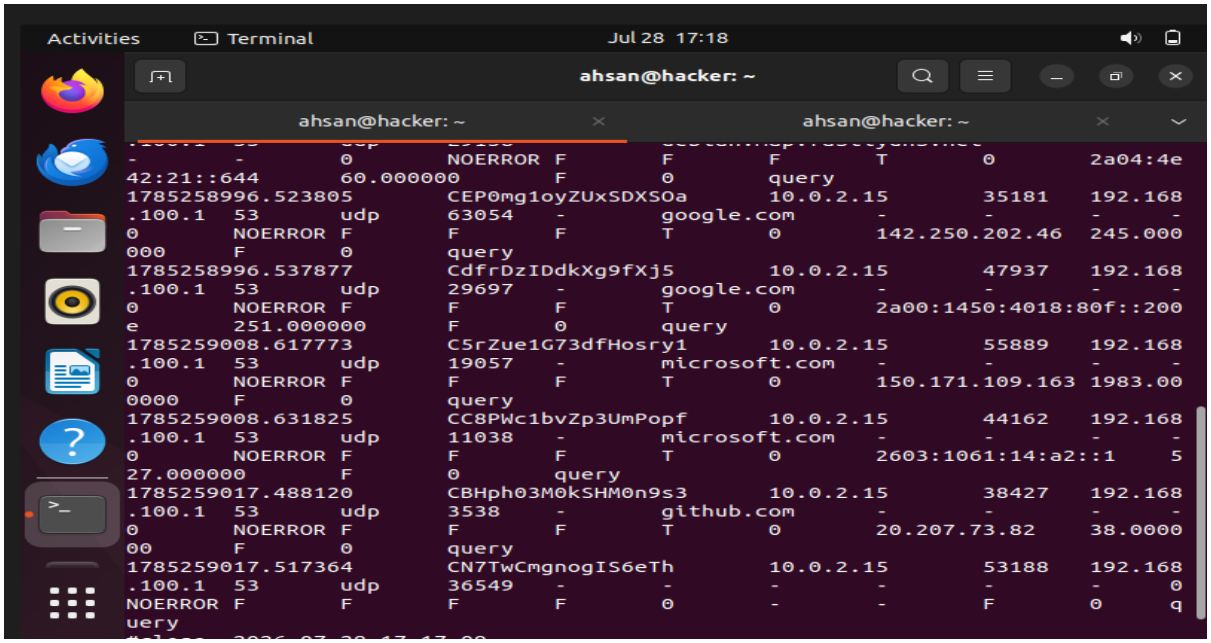
- **Requested resource: HTTP GET to example.com (port 80)**
- **Response: HTTP/200 OK**
- **Connection duration and byte counts captured in conn.log with service tagged as `http`**

Task Z2: Generate and Analyze DNS Traffic

Zeek was restarted for a clean capture. `dnsutils` was installed inside the container (minimal image lacked `nslookup`), then the following queries were run:

nslookup google.com
nslookup microsoft.com
nslookup github.com

***`dns.log` records every DNS query and response Zeek observes the queried domain, query type (A/AAAA/MX etc.), the response code , and the resolved IP addresses.**



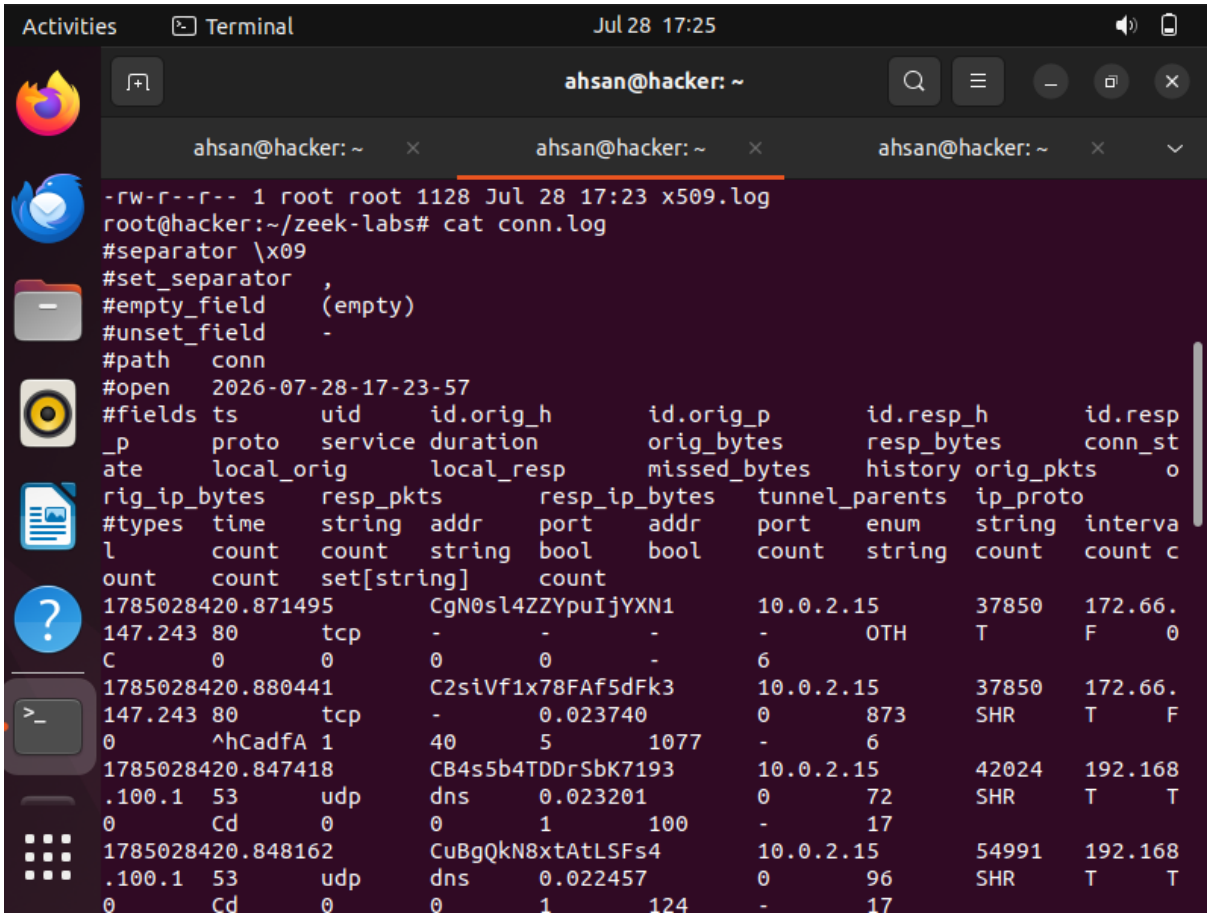
- **google.com → 142.250.202.46 (A) and 2a00:1450:4018:80f::200e**
- **microsoft.com → 150.171.109.163 and 2603:1061:14:a2::15**
- **github.com → 20.207.73.82**
- **All queries returned `NOERROR`, confirming successful resolution**

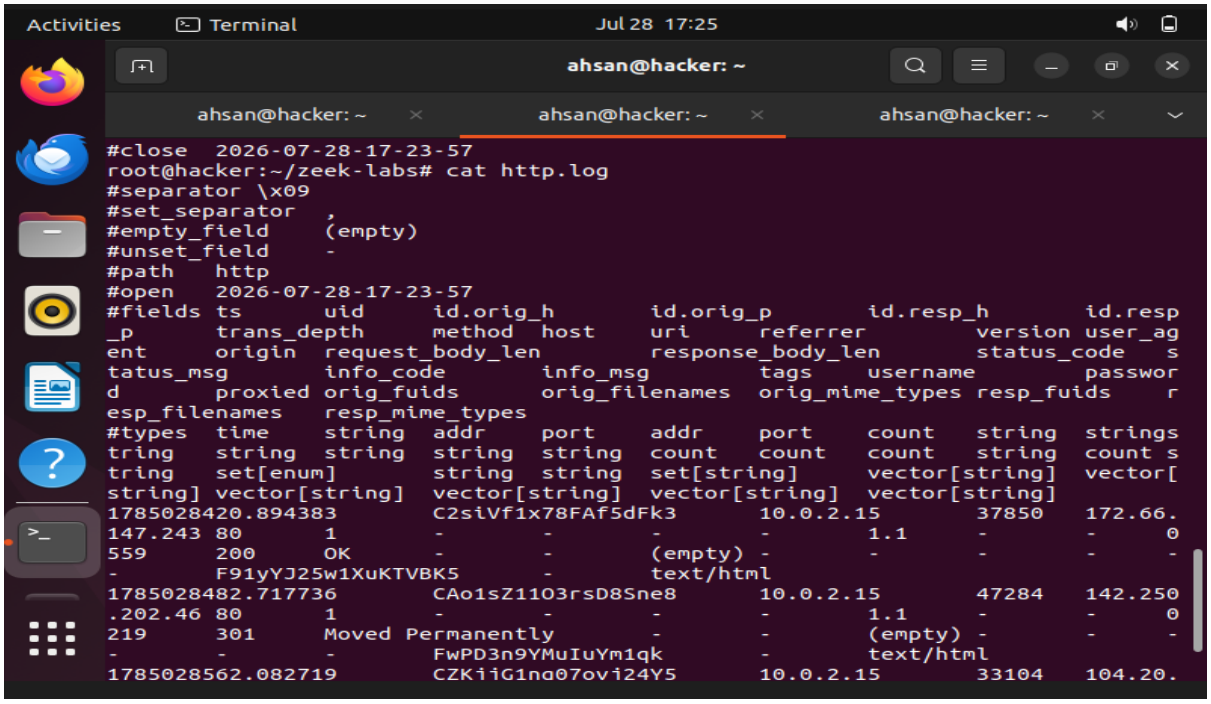
Task Z3: Analyze a PCAP File

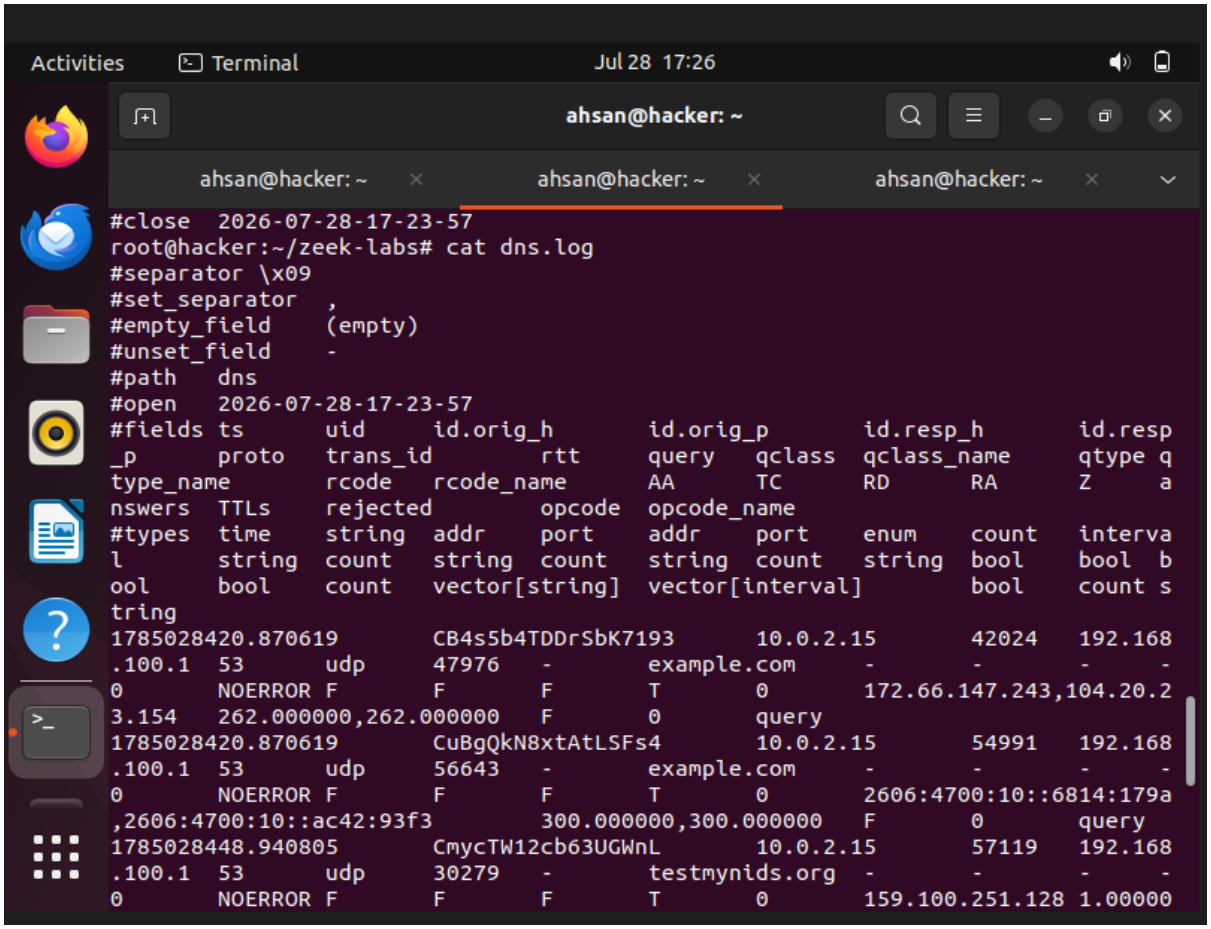
Rather than downloading a new sample, the `test.pcap` file created earlier with `tcpdump` for Suricata's PCAP test was reused. It was copied into the running container and analyzed offline:

sudo docker cp ~/test.pcap <container>:/root/zeek-labs/test.pcap
zeek -r test.pcap

***This demonstrates Zeek's core forensic use case analyzing previously captured traffic without needing a live interface. Zeek reconstructs the same structured logs (conn/http/dns) from stored packet data as it would from live traffic.**



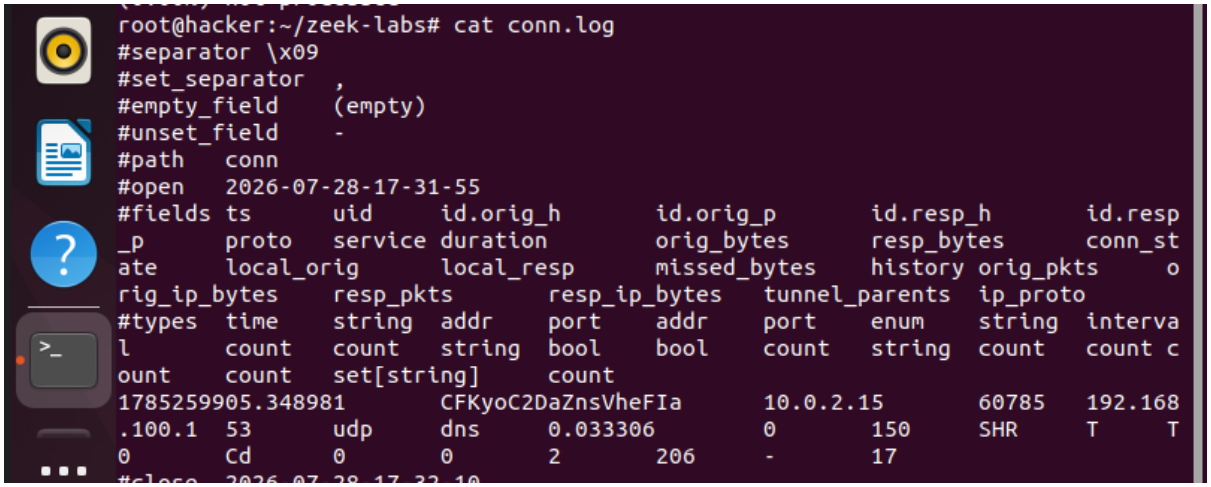

Activities
Terminal
Jul 28 17:25
ahsan@hacker: ~
ahsan@hacker: ~
ahsan@hacker: ~
ahsan@hacker: ~
#close 2026-07-28-17-23-57
root@hacker:~/zeek-labs# cat http.log
#separator \x09
#set_separator ,
#empty_field (empty)
#unset_field -
#path http
#open 2026-07-28-17-23-57
#fields ts uid id.orig_h id.orig_p id.resp_h id.resp_p trans_depth method host uri referrer version user_agent origin request_body_len response_body_len status_code status_msg info_code info_msg tags username password proxied orig_fuids orig_filenames orig_mime_types resp_fuids resp_filenames resp_mime_types
#types time string addr port addr port count string string string string string count count count string count string string set[enum] string string set[string] vector[string] vector[string] vector[string] vector[string]
1785028420.894383 C2siVf1x78FAf5dFk3 10.0.2.15 37850 172.66.147.243 80 1 - - - - 1.1 - - 0 559 200 OK - - (empty) - - - - - F91yYJ25w1XuKTVBK5 - text/html
1785028482.717736 CAo1sZ11O3rsD8Sne8 10.0.2.15 47284 142.250.202.46 80 1 - - - - 1.1 - - 0 219 301 Moved Permanently - - (empty) - - - - - FwPD3n9YMuIuYm1qk - text/html
1785028562.082719 CZKiiG1nq07ovi24Y5 10.0.2.15 33104 104.20.

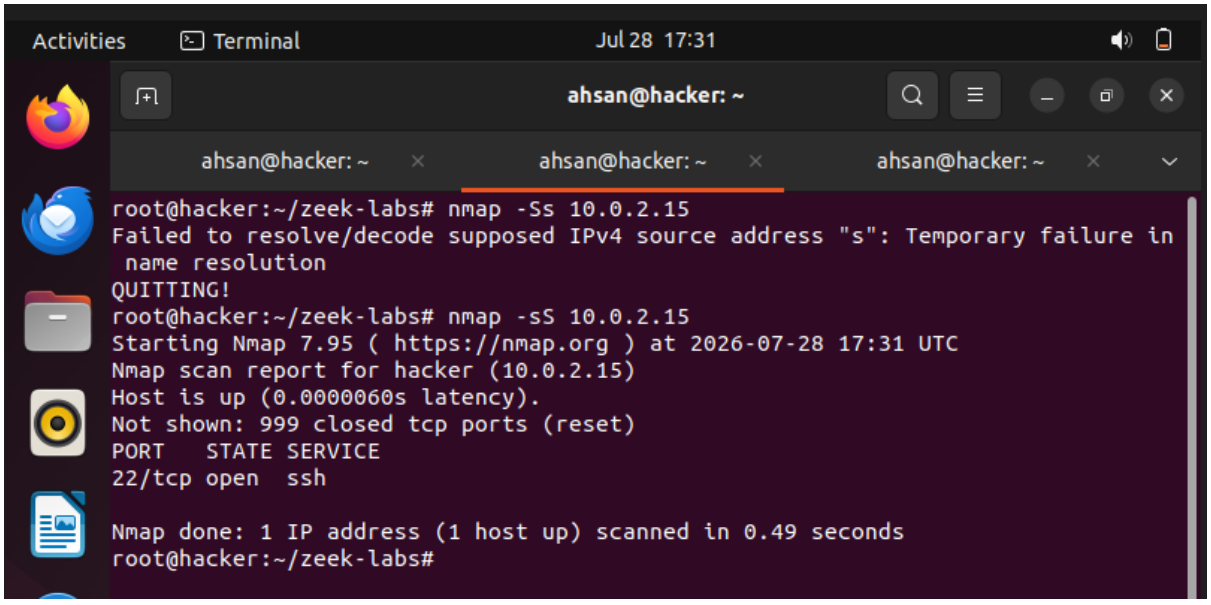

Activities
Terminal
Jul 28 17:26
ahsan@hacker: ~
ahsan@hacker: ~
ahsan@hacker: ~
ahsan@hacker: ~
#close 2026-07-28-17-23-57
root@hacker:~/zeek-labs# cat dns.log
#separator \x09
#set_separator ,
#empty_field (empty)
#unset_field -
#path dns
#open 2026-07-28-17-23-57
#fields ts uid id.orig_h id.orig_p id.resp_h id.resp_p proto trans_id rtt query qclass qclass_name qtype qtype_name rcode rcode_name AA TC RD RA Z answers TTLs rejected opcode opcode_name
#types time string addr port addr port enum count interval string count string count string count string bool bool bool bool count vector[string] vector[interval] bool count string
1785028420.870619 CB4s5b4TDDrSbK7193 10.0.2.15 42024 192.168.100.1 53 udp 47976 - example.com - - - - 0 NOERROR F F F T 0 172.66.147.243,104.20.23.154 262.000000,262.000000 F 0 query
1785028420.870619 CuBgQkN8xtAtLSFs4 10.0.2.15 54991 192.168.100.1 53 udp 56643 - example.com - - - - 0 NOERROR F F F T 0 2606:4700:10::6814:179a,2606:4700:10::ac42:93f3 300.000000,300.000000 F 0 query
1785028448.940805 CmycTW12cb63UGWnL 10.0.2.15 57119 192.168.100.1 53 udp 30279 - testmynids.org - - - - 0 NOERROR F F F T 0 159.100.251.128 1.00000

Task Z4: Observe Port Scan Behavior

nmap was installed inside the container. A SYN scan was run against `scanme.nmap.org` Nmap's official, legal public test target since scanning the container's own host IP produced no visible traffic on `enp0s3`.

zeek -i enp0s3
nmap -sS scanme.nmap.org





- **Command to count unique destination ports: `grep -v '^#' conn.log | awk '{print $7}' | sort -u | wc -l`**

Notes / Deviations from the Manual

- Standard apt based Zeek installation (OpenSUSE repository) failed with package resolution errors the Docker-based method from Section 14 was used instead.
- The Zeek Docker image is minimal and required manual installation of `curl`, `dnsutils`, and `nmap` inside the container to complete the practice tasks.
- For Z4, scanning the container's own host IP produced no visible traffic on `enp0s3` `scanme.nmap.org` was used instead as it is Nmap's official public test target.